

## 1.1 Review of Linear Algebra for machine learning

Linear algebra is essential for many machine learning algorithms and techniques. It helps in manipulating and processing data, which is often represented as vectors and matrices. These mathematical tools make computations faster and reveal patterns within the data.

It simplifies complex tasks like data transformation, dimensionality reduction (e.g., PCA), and optimization. Key concepts like matrix multiplication, eigenvalues, and linear transformations help in training models and improving predictions efficiently.

### Fundamental Concepts in Linear Algebra for Machine Learning

In machine learning, vectors, matrices, and scalars play key roles in handling and processing data.

Vectors are used to represent individual data points, where each number in the vector corresponds to a specific features of the dataset (like age, income, or hours ).

Matrices are considered as data storage units used to store large datasets, with rows representing different data points and columns representing features.

Scalars are single numbers that scale vectors or matrices, often used in algorithms like gradient descent to adjust the weights or learning rate, helping the model improve over time.

Common linear transformations in machine learning are operations that help manipulate data in useful ways, making it easier for models to learn patterns and make predictions. Some common linear transformations are:

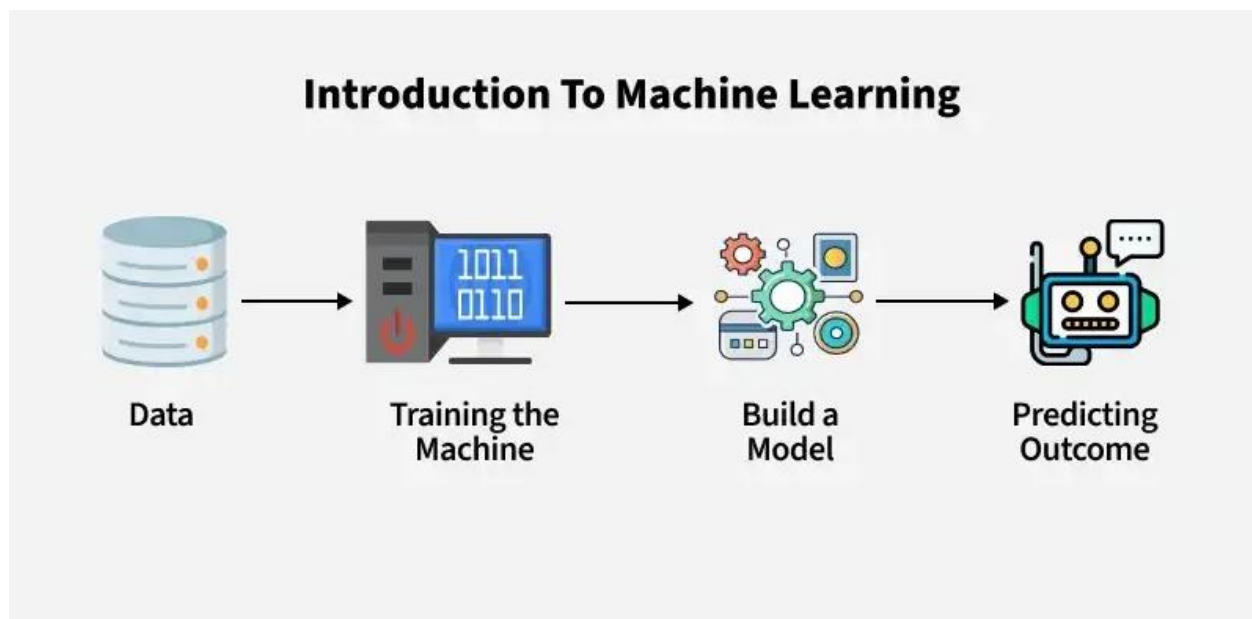
**Translation:** Translation means moving data points around without changing their shape or size. In machine learning, this is often used to center data by subtracting the average value from each data point.

**Scaling:** Scaling involves stretching or compressing vectors along each dimension. It is used in feature scaling to make sure all features are on a similar scale, so one feature doesn't dominate the model.

**Rotation:** Rotation involves turning data around a point or axis. It's not used much in basic machine learning but can be helpful in fields like computer vision and robotics.

## 1.2 Introduction and motivation for machine learning

**Machine learning (ML) allows computers to learn and make decisions without being explicitly programmed. It involves feeding data into algorithms to identify patterns and make predictions on new data.** Machine learning is used in various applications, including image and speech recognition, natural language processing, and recommender systems.



**Why do we need Machine Learning?**

**Machine Learning** algorithm learns from data, train on patterns, and solve or predict complex problems beyond the scope of traditional programming. It drives better decision-making and tackles intricate challenges efficiently.

Here's why ML is indispensable across industries:

### **1. Solving Complex Business Problems**

Traditional programming struggles with tasks like image recognition, natural language processing (NLP), and medical diagnosis. ML, however, thrives by learning from examples and making predictions without relying on predefined rules.

#### **Example Applications:**

- Image and speech recognition in healthcare.
- Language translation and sentiment analysis.

### **2. Handling Large Volumes of Data**

With the internet's growth, the data generated daily is immense. ML effectively processes and analyzes this data, extracting valuable insights and enabling real-time predictions.

#### **Use Cases:**

- Fraud detection in financial transactions.
- Social media platforms like Facebook and Instagram predicting personalized feed recommendations from billions of interactions.

### **3. Automate Repetitive Tasks**

ML automates time-intensive and repetitive tasks with precision, reducing manual effort and error-prone systems.

#### **Examples:**

- **Email Filtering:** Gmail uses ML to keep your inbox spam-free.
- **Chatbots:** ML-powered chatbots resolve common issues like order tracking and password resets.
- **Data Processing:** Automating large-scale invoice analysis for key insights.

### **4. Personalized User Experience**

ML enhances user experience by tailoring recommendations to individual preferences. Its algorithms analyze user behavior to deliver highly relevant content.

#### **Real-World Applications:**

- **Netflix:** Suggests movies and TV shows based on viewing history.

- **E-Commerce:** Recommends products you're likely to purchase.

## 5. Self Improvement in Performance

ML models evolve and improve with more data, making them smarter over time. They adapt to user behavior and refine their performance.

### Examples:

- **Voice Assistants** (e.g., Siri, Alexa): Learn user preferences, improve voice recognition, and handle diverse accents.
- **Search Engines:** Refine ranking algorithms based on user interactions.
- **Self-Driving Cars:** Enhance decision-making using millions of miles of data from simulations and real-world driving.

### What Makes a Machine “Learn”?

A machine “learns” by recognizing patterns and improving its performance on a task based on data, without being explicitly programmed.

The process involves:

1. **Data Input:** Machines require data (e.g., text, images, numbers) to analyze.
2. **Algorithms:** Algorithms process the data, finding patterns or relationships.
3. **Model Training:** Machines learn by adjusting their parameters based on the input data using mathematical models.
4. **Feedback Loop:** The machine compares predictions to actual outcomes and corrects errors (via optimization methods like gradient descent).
5. **Experience and Iteration:** Repeating this process with more data improves the machine's accuracy over time.
6. **Evaluation and Generalization:** The model is tested on unseen data to ensure it performs well on real-world tasks.

In essence, machines “learn” by continuously refining their understanding through data-driven iterations, much like humans learn from experience.

### Importance of Data in Machine Learning

Data is the foundation of machine learning (ML). Without quality data, ML models cannot learn, perform, or make accurate predictions.

- Data provides the examples from which models learn patterns and relationships.
- High-quality and diverse data improves model accuracy and generalization.

- Data ensures models understand real-world scenarios and adapt to practical applications.
- Features derived from data are critical for training models.
- Separate datasets for validation and testing assess how well the model performs on unseen data.
- Data fuels iterative improvements in ML models through feedback loops.

## Types of Machine Learning

### 1. Supervised learning

**Supervised learning** is a type of machine learning where a model is trained on labeled data—meaning each input is paired with the correct output. The model learns by comparing its predictions with the actual answers provided in the training data.

Both *classification* and *regression* problems are supervised learning problems.

**Example:** Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients and each patient is labeled as “healthy” or “sick”.

| Gender | Age | Label   |
|--------|-----|---------|
| M      | 48  | sick    |
| M      | 67  | sick    |
| F      | 53  | healthy |
| M      | 49  | sick    |
| F      | 32  | healthy |
| M      | 34  | healthy |
| M      | 21  | healthy |

In this example, supervised learning is to use this labeled data to train a model that can predict the label (“healthy” or “sick”) for new patients based on their gender and age. For instance, if a

new patient (e.g., Male, 50 years old) visits the clinic, the model can classify whether the patient is “healthy” or “sick” based on the patterns it learned during training.

## 2. Unsupervised learning:

**Unsupervised learning** algorithms draw inferences from datasets consisting of input data without labeled responses. In unsupervised learning algorithms, classification or categorization is not included in the observations.

**Example:** Consider the following data regarding patients entering a clinic. The dataset includes **unlabeled data**, where only the gender and age of the patients are available, with no health status labels.

| Gender | Age |
|--------|-----|
| M      | 48  |
| M      | 67  |
| F      | 53  |
| M      | 49  |
| F      | 34  |
| M      | 21  |

Here, unsupervised learning technique will be used to find patterns or groupings in the data such as clustering patients by age or gender. For example, the algorithm might group patients into clusters, such as “younger healthy patients” or “older patients,” without prior knowledge of their health status.

## 3. Reinforcement Learning

**Reinforcement Learning (RL)** trains an agent to act in an environment by maximizing rewards through trial and error. Unlike other machine learning types, RL doesn’t provide explicit instructions.

Instead, the agent learns by:

- **Exploring Actions:** Trying different actions.
- **Receiving Feedback:** Rewards for correct actions, punishments for incorrect ones.
- **Improving Performance:** Refining strategies over time.

**Example:** Identifying a Fruit

The system receives an input (e.g., an apple) and initially makes an incorrect prediction (“It’s a mango”). Feedback is provided to correct the error (“Wrong! It’s an apple”), and the system updates its model based on this feedback.

Over time, it learns to respond correctly (“It’s an apple”) when encountering similar inputs, improving accuracy through trial, error, and feedback.



Beyond these three of machine learning techniques, there are two additional approaches have gained significant attention in modern Machine Learning **Self-Supervised Learning** and **Semi-Supervised Learning**.

### 1.3 Examples of machine learning applications

Machine learning is one of the most exciting technologies that one would have ever come across. As is evident from the name, it gives the computer that which makes it more similar to humans: The ability to learn. Machine learning is actively being used today, perhaps in many more places than one would expect.

Some of the most common examples are:

- Image Recognition
- Speech Recognition
- Recommender Systems
- Fraud Detection
- Self Driving Cars
- Medical Diagnosis
- Stock Market Trading
- Virtual Try On

### **Image Recognition**

Image Recognition is one of the reasons behind the boom one could have experienced in the field of Deep Learning. The task which started from classification between cats and dog images has now evolved up to the level of Face Recognition and real-world use cases based on that like employee attendance tracking.

Also, image recognition has helped revolutionized the healthcare industry by employing smart systems in disease recognition and diagnosis methodologies.

### **Speech Recognition**

Speech Recognition based smart systems like Alexa and Siri have certainly come across and used to communicate with them. In the backend, these systems are based basically on Speech Recognition systems. These systems are designed such that they can convert voice instructions into text.

One more application of the Speech recognition that we can encounter in our day-to-day life is that of performing Google searches just by speaking to it.

### **Recommender Systems**

As our world has digitalized more and more approximately every tech giants try to provide customized services to its users. This application is possible just because of the recommender systems which can analyze a user's preferences and search history and based on that they can recommend content or services to them.

An example of these services is very common for example youtube. It recommends new videos and content based on the user's past search patterns. Netflix recommends movies and series based on the interest provided by users when someone creates an account for the very first time.

### **Fraud Detection**

In today's world, most things have been digitalized varying from buying toothbrushes or making transactions of millions of dollars everything is accessible and easy to use. But with this process of digitization cases of fraudulent transactions and fraudulent activities have increased. Identifying them is not that easy but machine learning systems are very efficient in these tasks. Due to these applications only whenever the system detects red flags in a user's activity than a suitable notification be provided to the administrator so, that these cases can be monitored properly for any spam or fraud activities.

### **Self Driving Cars**

It would have been assumed that there is certainly some ghost who is driving a car if we ever saw a car being driven without a driver but all thanks to machine learning and deep learning that in today's world, this is possible and not a story from some fictional book. Even though the algorithms and tech stack behind these technologies are highly advanced but at the core it is machine learning which has made these applications possible.

The most common example of this use case is that of the Tesla cars which are well-tested and proven for autonomous driving.

### **Medical Diagnosis**

If you are a machine learning practitioner or even if you are a student then you must have heard about projects like breast cancer Classification, Parkinson's Disease Classification, Pneumonia detection, and many more health-related tasks which are performed by machine learning models with more than 90% of accuracy.

Not even in the field of disease diagnosis in human beings but they work perfectly fine for plant disease-related tasks whether it is to predict the type of disease it is or to detect whether some disease is going to occur in the future.

### **Stock Market Trading**

Stock Market has remained a hot topic among working professionals and even students because if you have sufficient knowledge of the markets and the forces which drives them then you can make fortune in this domain. Attempts have been made to create intelligent systems which can predict future price trends and market value as well.

This can be considered as one of the applications of time series forecasting because stock price data is nothing but sequential data in which the time at which data has been taken is of utmost importance

#### **1.4 Vapnik-Chervonenkis (VC) dimension**

The Vapnik-Chervonenkis (VC) dimension is a measure of the capacity of a hypothesis set to fit different data sets. It was introduced by Vladimir Vapnik and Alexey Chervonenkis in the 1970s and has become a fundamental concept in statistical learning theory. The VC dimension is a measure of the complexity of a model, which can help us understand how well it can fit different data sets.

The VC dimension of a hypothesis set  $H$  is the largest number of points that can be shattered by  $H$ . A hypothesis set  $H$  shatters a set of points  $S$  if, for every possible labeling of the points in  $S$ , there exists a hypothesis in  $H$  that correctly classifies the points. In other words, a hypothesis set shatters a set of points if it can fit any possible labeling of those points.

#### **Bounds of VC – Dimension**

The VC dimension provides both upper and lower bounds on the number of training examples required to achieve a given level of accuracy. The upper bound on the number of training examples is logarithmic in the VC dimension, while the lower bound is linear.

#### **Applications of VC – Dimension**

The VC dimension has a wide range of applications in machine learning and statistics. For example, it is used to analyze the complexity of neural networks, support vector machines, and decision trees. The VC dimension can also be used to design new learning algorithms that are robust to noise and can generalize well to unseen data.

The VC dimension can be extended to more complex learning scenarios, such as multiclass classification and regression. The concept of the VC dimension can also be applied to other areas of computer science, such as computational geometry and graph theory.

## Code Implementation for VC – Dimension

The VC dimension is a theoretical concept that cannot be directly computed from data. However, we can estimate the VC dimension for a given hypothesis set by counting the number of points that can be shattered by the set. In Python, we can implement a function that computes the VC dimension of a given hypothesis set using this approach.

The function takes a hypothesis set as its input and computes the VC dimension using the brute-force approach of checking all possible combinations of points and labels. It uses the itertools module to generate all possible combinations of points and labels and then checks if the hypothesis set can shatter each combination. The function returns the estimated VC dimension of the hypothesis set.

Let's illustrate the usage of this function with some examples:

### Example 1:

Suppose we have a hypothesis set that consists of all linear functions of form  $f(x) = ax + b$ , where  $a$  and  $b$  are real numbers. We can define this hypothesis set in Python as follows:

- Python

```
import itertools
```

```
def vc_dimension(hypothesis_set):
```

```
    """
```

```
    Estimates the VC dimension of a hypothesis set using the brute-force approach.
```

```
"""
```

```
n = 4
```

```
while True:
```

```
    points = [(i, j) for i in range(n) for j in range(2)]
```

```
    shattered_sets = 0
```

```
    for combination in itertools.combinations(points, n):
```

```
        is_shattered = True
```

```
        for labeling in itertools.product([0, 1], repeat=n):
```

```
            hypotheses = [hypothesis_set(point) for point in combination]
```

```
            if set(hypotheses) != set(labeling):
```

```
                is_shattered = False
```

```
                break
```

```
        if is_shattered:
```

```
            shattered_sets += 1
```

```
    else:
```

```
        break

    if not is_shattered:

        break

    n += 1

return n-1 if shattered_sets == 2**n else n-2


# Example 1: linear function hypothesis set

def linear_function(point):

    x, y = point

    return int(y >= x)


print(vc_dimension(linear_function))
```

**Output:**

2

In example 1, the `linear_function` function implements a simple linear function hypothesis set that returns 1 if the y-coordinate of the input point is greater than or equal to the x-coordinate, and 0 otherwise. The `vc_dimension` function is then used to estimate the VC dimension of this hypothesis set, which is 2.

## 1.5 Probably Approximately Correct (PAC) learning

PAC learning is a theoretical framework that addresses the question of how much data is necessary for a learning algorithm to perform well on new, unseen data. The core idea is that a learning algorithm can be considered PAC if, given a sufficient number of training samples, it can produce a hypothesis that is likely (with high probability) to be approximately correct (within a specified error margin).

### Key Components of PAC Learning

1. **Hypothesis Space:** This is the set of all possible hypotheses that a learning algorithm can choose from. The complexity of the hypothesis space significantly impacts the sample complexity required for learning.
2. **Sample Complexity:** This refers to the number of training examples needed to ensure that the learned hypothesis will generalize well to new data. In PAC learning, it is crucial to determine how many samples are required to achieve a desired level of accuracy and confidence.
3. **Generalization:** This is the ability of a learning algorithm to perform well on unseen data. In the PAC framework, generalization is quantified by the probability that the chosen hypothesis will have an error rate within an acceptable range on new samples.
4. **Error Rate:** The error rate is defined as the probability that the hypothesis will misclassify an example drawn from the underlying distribution. PAC learning aims to minimize this error rate while ensuring that the hypothesis is consistent with the training data.

### Challenges in PAC Learning

Despite its advantages, PAC learning faces several challenges:

- **Computational Complexity:** Finding the optimal hypothesis can be computationally expensive, especially as the hypothesis space grows.

- **Model Assumptions:** PAC learning relies on certain assumptions about the underlying distribution of the data, which may not always hold in practice.
- **Overfitting:** As the complexity of the hypothesis space increases, there is a risk of overfitting, where the model performs well on training data but poorly on unseen data.

### Applications of PAC Learning

PAC learning has broad applications across various domains:

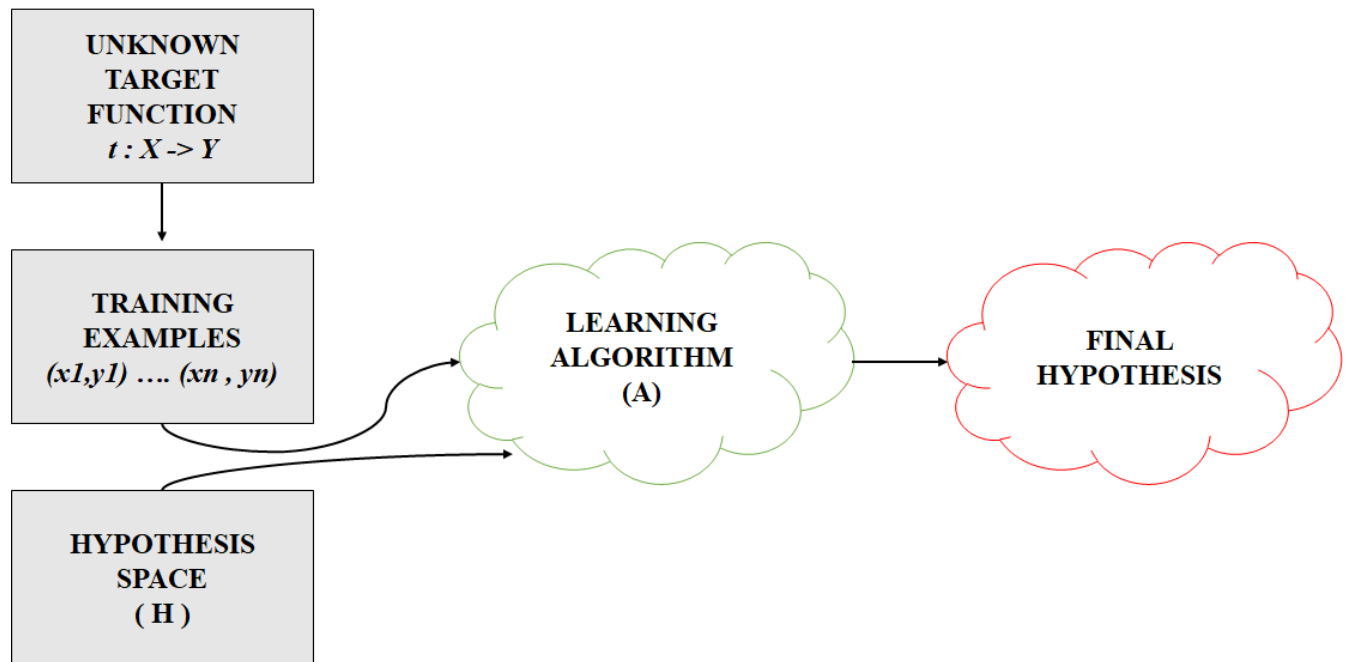
- **Classification:** It serves as a foundation for designing classifiers that can generalize well from limited training data.
- **Active Learning:** PAC learning principles guide the selection of the most informative samples to label, minimizing the sample complexity.
- **Reinforcement Learning:** The framework helps in understanding the trade-offs between exploration and exploitation in learning environments.

### 1.6 Hypothesis spaces

**A hypothesis in machine learning is the model's presumption regarding the connection between the input features and the result.** It is an illustration of the mapping function that the algorithm is attempting to discover using the training set. To minimize the discrepancy between the expected and actual outputs, the learning process involves modifying the weights that parameterize the hypothesis. The objective is to optimize the model's parameters to achieve the best predictive performance on new, unseen data, and a cost function is used to assess the hypothesis' accuracy.

In most supervised machine learning algorithms, our main goal is to find a possible hypothesis from the hypothesis space that could map out the inputs to the proper outputs. The following figure shows the common method to find out the possible hypothesis from the Hypothesis space:

## PRODUCING FINAL HYPOTHESIS



### Hypothesis Space (H)

Hypothesis space is the set of all the possible legal hypothesis. This is the set from which the machine learning algorithm would determine the best possible (only one) which would best describe the target function or the outputs.

### Hypothesis (h)

A hypothesis is a function that best describes the target in supervised machine learning. The hypothesis that an algorithm would come up depends upon the data and also depends upon the restrictions and bias that we have imposed on the data.

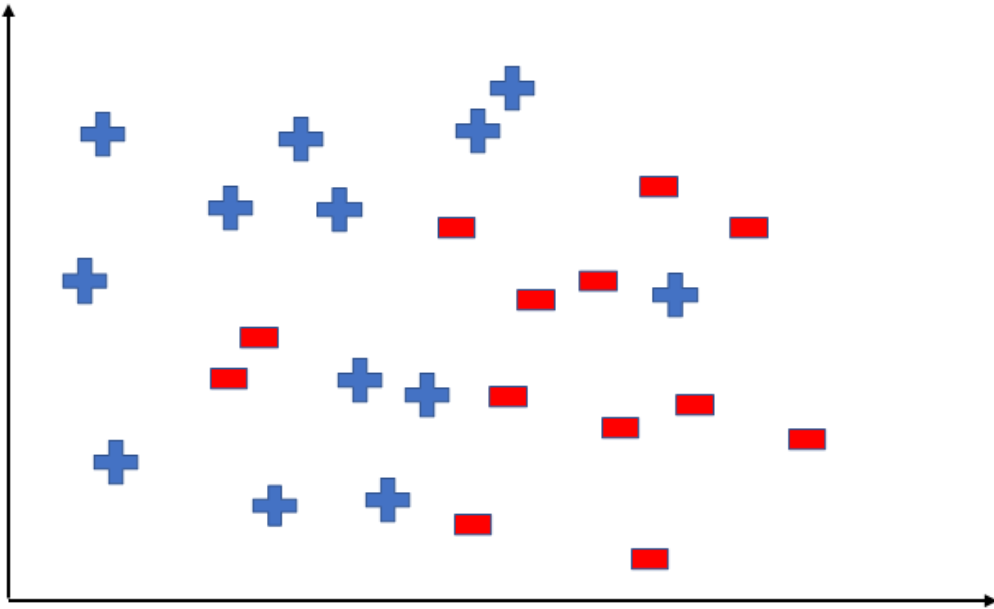
The Hypothesis can be calculated as:

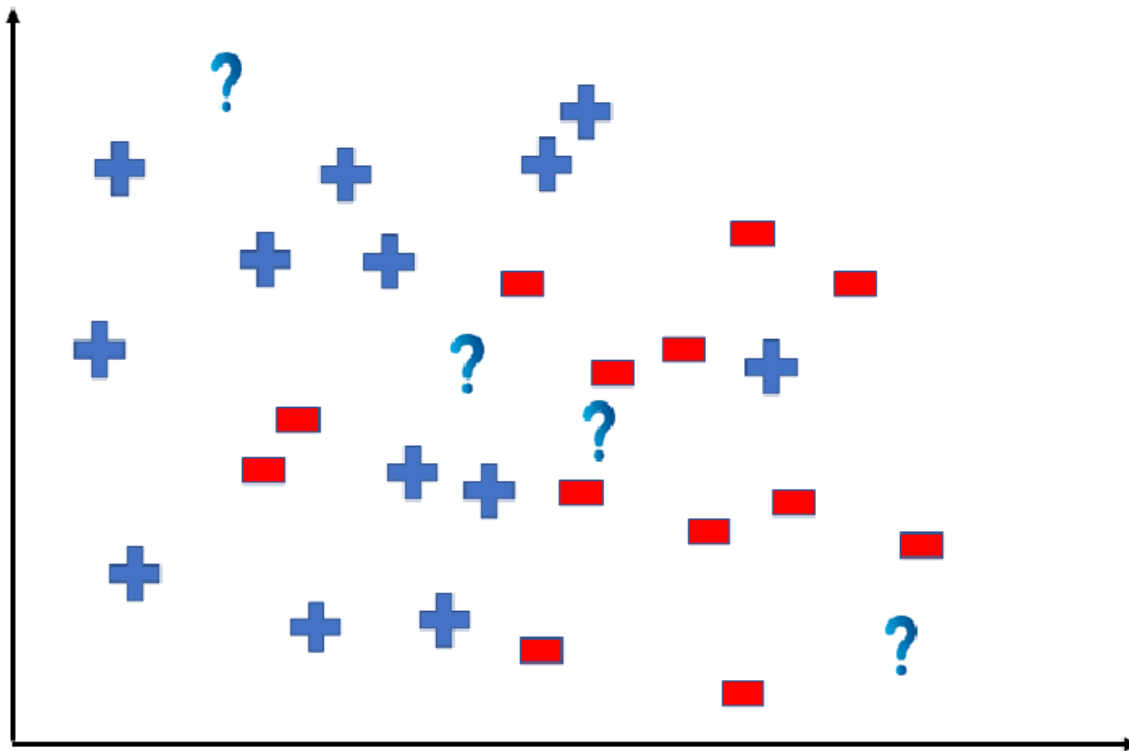
$$y = mx + b$$

Where,

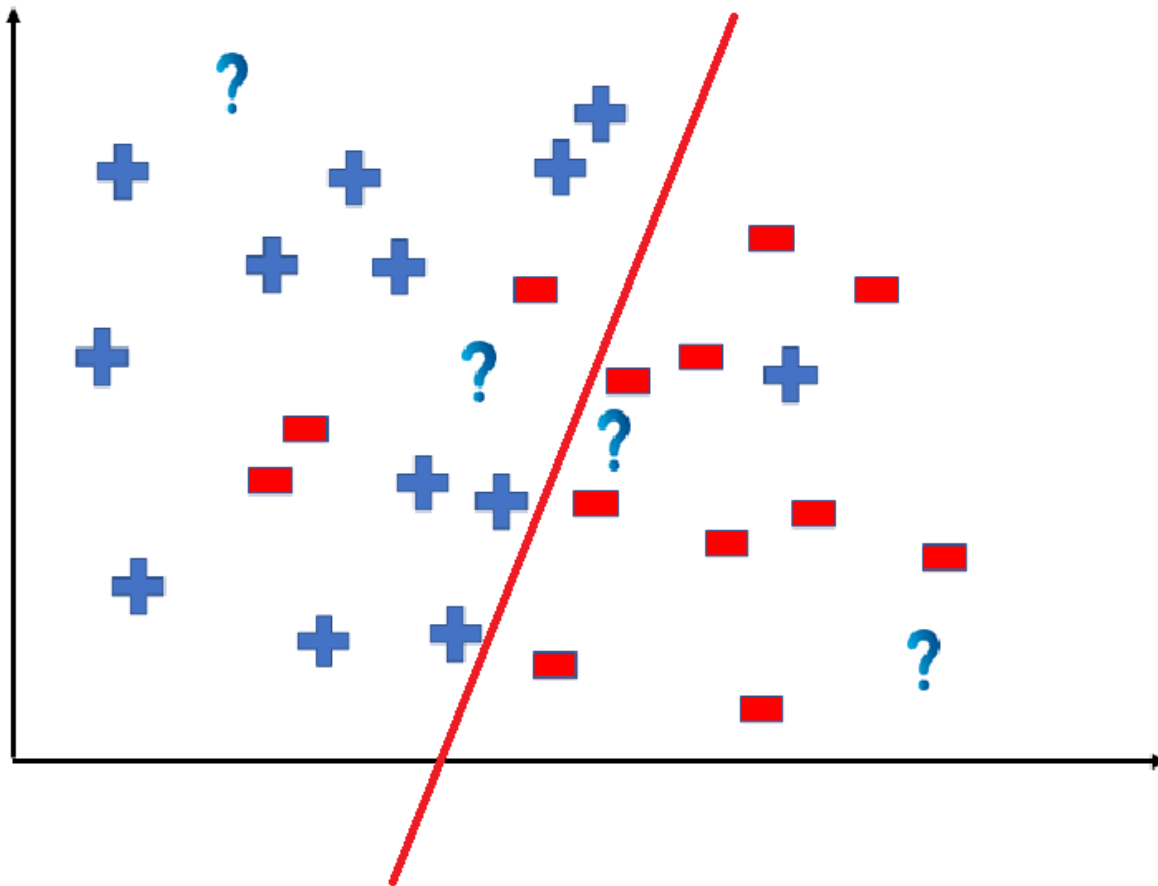
- $y$  = range
- $m$  = slope of the lines
- $x$  = domain
- $b$  = intercept

To better understand the Hypothesis Space and Hypothesis consider the following coordinate that shows the distribution of some data:

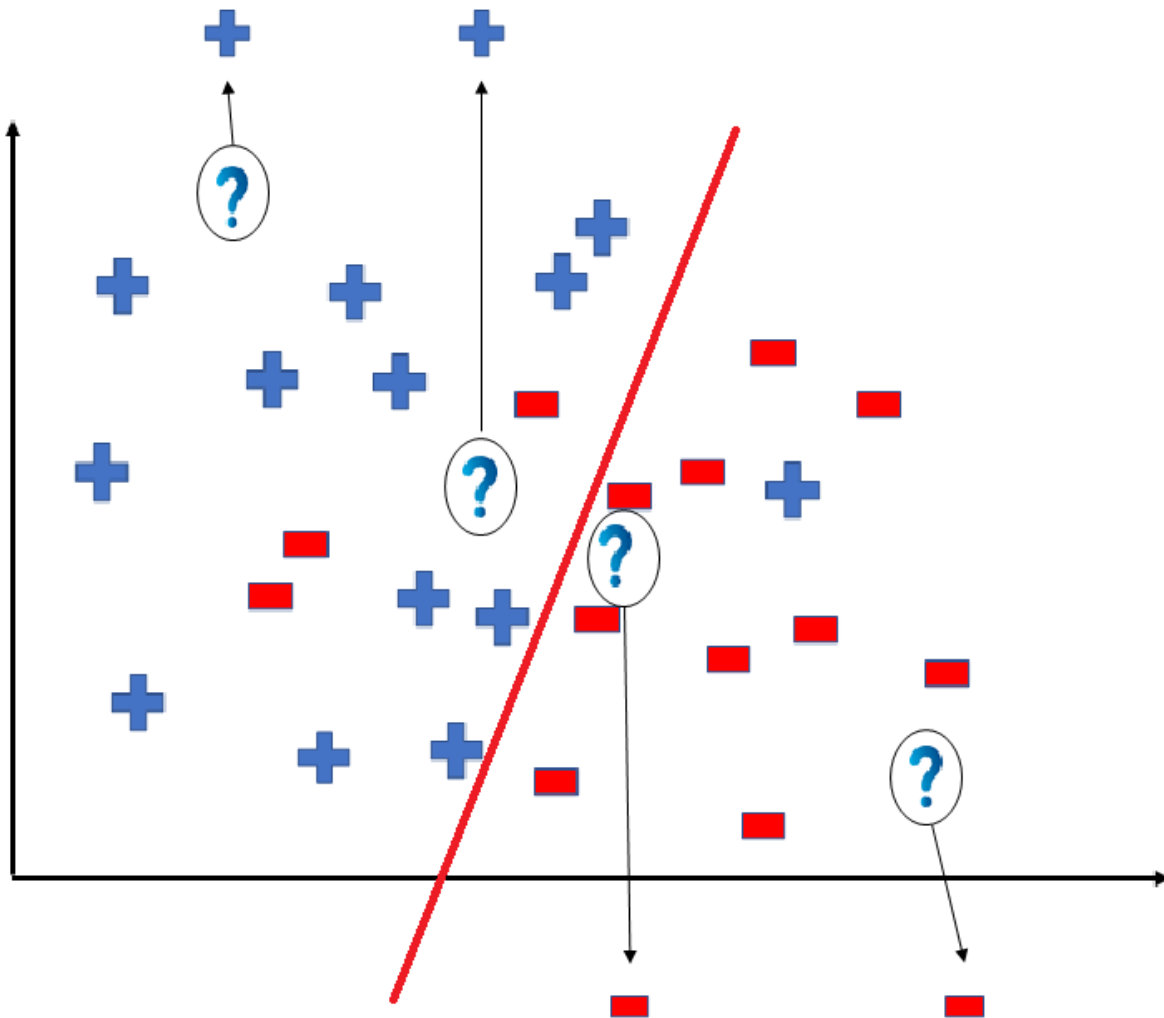




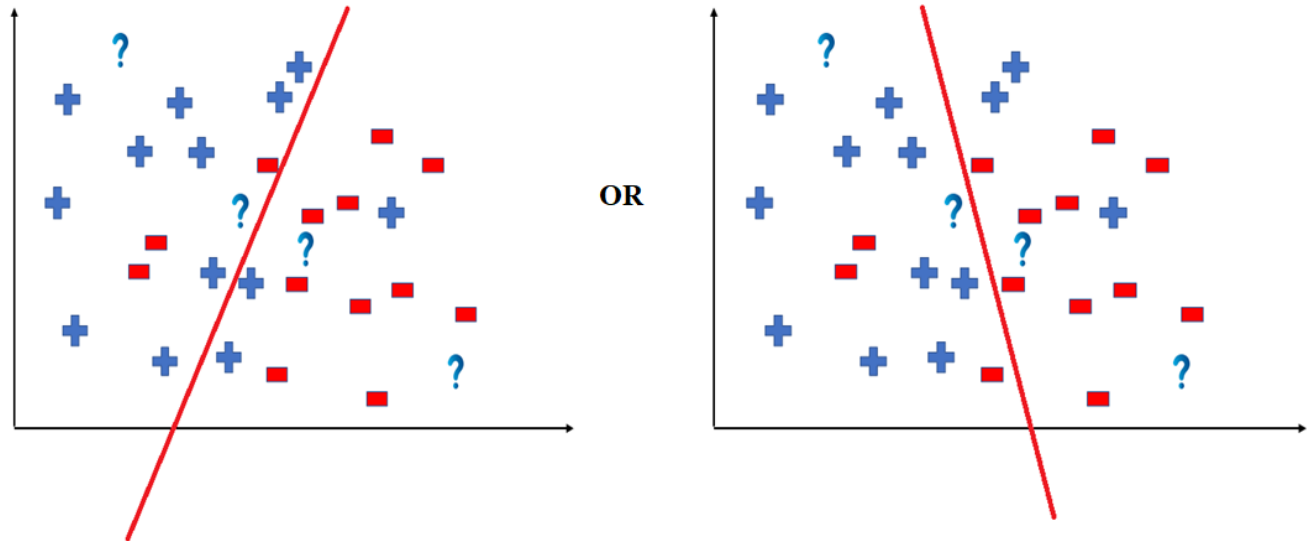
We can predict the outcomes by dividing the coordinate as shown below:



So the test data would yield the following result:



But note here that we could have divided the coordinate plane as:



The way in which the coordinate would be divided depends on the data, algorithm and constraints.

- All these legal possible ways in which we can divide the coordinate plane to predict the outcome of the test data composes of the Hypothesis Space.
- Each individual possible way is known as the hypothesis.

## 1.7 Inductive bias

Inductive bias can be defined as the set of assumptions or biases that a learning algorithm employs to make predictions on unseen data based on its training data. These assumptions are inherent in the algorithm's design and serve as a foundation for learning and generalization.

The inductive bias of an algorithm influences how it selects a hypothesis (a possible explanation or model) from the hypothesis space (the set of all possible hypotheses) that best fits the training data. It helps the algorithm navigate the trade-off between fitting the training data perfectly (overfitting) and generalizing well to unseen data (underfitting).

### Types of Inductive Bias

Inductive bias can manifest in various forms, depending on the algorithm and its underlying assumptions. Some common types of inductive bias include:

1. **Bias towards simpler explanations:** Many machine learning algorithms, such as decision trees and linear models, have a bias towards simpler hypotheses. They prefer explanations that are more parsimonious and less complex, as these are often more likely to generalize well to unseen data.
2. **Bias towards smoother functions:** Algorithms like kernel methods or Gaussian processes have a bias towards smoother functions. They assume that neighboring points in the input space should have similar outputs, leading to smooth decision boundaries.
3. **Bias towards specific types of functions:** Neural networks, for example, have a bias towards learning complex, nonlinear functions. This bias allows them to capture intricate patterns in the data but can also lead to overfitting if not regularized properly.
4. **Bias towards sparsity:** Some algorithms, like Lasso regression, have a bias towards sparsity. They prefer solutions where only a few features are relevant, which can improve interpretability and generalization.

### **Importance of Inductive Bias**

Inductive bias is crucial in machine learning as it helps algorithms generalize from limited training data to unseen data. Without a well-defined inductive bias, algorithms may struggle to make accurate predictions or may overfit the training data, leading to poor performance on new data.

Understanding the inductive bias of an algorithm is essential for model selection, as different biases may be more suitable for different types of data or tasks. It also provides insights into how the algorithm is learning and what assumptions it is making about the data, which can aid in interpreting its predictions and results.

## **1.8 Generalization**

Generalization in machine learning refers to the ability of a trained model to accurately make predictions on new, unseen data. The purpose of generalization is to equip the model to understand the patterns and relationships within its training data and apply them to previously unseen examples from within the same distribution as the training set. Generalization is foundational to

the practical usefulness of machine learning and deep learning algorithms because it allows them to produce models that can make reliable predictions in real-world scenarios.

Generalization is important because the true test of a model's effectiveness is not how well it performs on the training data, but rather how well it generalizes to new and unseen data. If a model fails to generalize, it may exhibit high accuracy on the training set but will likely perform poorly on real-world examples. This limitation renders the model impractical and unreliable in practical applications.

A spam email classifier is a great example of generalization in machine learning. Suppose you have a training dataset containing emails labeled as either *spam* or *not spam* and your goal is to build a model that can accurately classify incoming emails as spam or legitimate based on their content.

During the training phase, the machine learning algorithm learns from the set of labeled emails, extracting relevant features and patterns to make predictions. The model optimizes its parameters to minimize the training error and achieve high accuracy on the training data.

Now, the true test of the model's effectiveness lies in its ability to generalize to new, unseen emails. When new emails arrive, the model needs to accurately classify them as spam or legitimate without prior exposure to their content. This is where generalization comes in.

In this case, generalization enables the model to identify the underlying patterns and characteristics that distinguish spam from legitimate emails. It allows the model to generalize its learned knowledge beyond the specific examples in the training set and apply it to unseen data.

Without generalization, the model may become too specific to the training set, memorizing specific words or phrases that were common in the training data and failing to understand new examples. As a result, the model could incorrectly classify legitimate emails as spam or fail to detect new spam patterns.

## Overfitting and Underfitting:

To achieve successful generalization, machine learning practitioners must address challenges like overfitting and underfitting.

- **Overfitting** refers to a scenario where a machine learning model memorizes the training data but does not correctly learn its underlying patterns. Overfit models perform exceptionally well on training data but fail to generalize to new, unseen data. This is because the model becomes too complex or too specialized to the training set, capturing noise, outliers, or random fluctuations in the data as meaningful patterns. Overfitting causes the model to be overly sensitive to small fluctuations in the training data, making it less robust to noise or variations in the real world.
- **Underfitting** occurs when a machine learning model is too simplistic and can't capture the underlying patterns in the data. An underfit model typically exhibits high error on both the training and testing data. Underfit models also typically exhibit high bias because they're not expressive enough to accurately represent the data.

To avoid overfitting and underfitting, the selection of the appropriate algorithm is important because. Here are examples of algorithms and their tendencies toward overfitting or underfitting:

**Decision Trees** have a high capacity to capture intricate details and noise in training data – creating complex, deep trees – which can lead to overfitting. Techniques like pruning, setting a maximum tree depth, or applying regularization methods can help prevent overfitting in decision trees.

### 1.9 Bias variance trade-off

The bias is known as the difference between the prediction of the values by the Machine Learning model and the correct value. Being high in biasing gives a large error in training as well as testing data. It is recommended that an algorithm should always be low-biased to avoid the problem of underfitting. By high bias, the data predicted is in a straight line format, thus not fitting accurately in the data in the data set. Such fitting is known as the **Underfitting of Data**.

This happens when the hypothesis is too simple or linear in nature. Refer to the graph given below for an example of such a situation.

The variability of model prediction for a given data point which tells us the spread of our data is called the variance of the model. The model with high variance has a very complex fit to the training data and thus is not able to fit accurately on the data which it hasn't seen before. As a result, such models perform very well on training data but have high error rates on test data. When a model is high on variance, it is then said to as **Overfitting of Data**. Overfitting is fitting the training set accurately via complex curve and high order hypothesis but is not the solution as the error with unseen data is high. While training a data model variance should be kept low.

If the algorithm is too simple (hypothesis with linear equation) then it may be on high bias and low variance condition and thus is error-prone. If algorithms fit too complex (hypothesis with high degree equation) then it may be on high variance and low bias. In the latter condition, the new entries will not perform well. Well, there is something between both of these conditions, known as a Trade-off or Bias Variance Trade-off. This tradeoff in complexity is why there is a tradeoff between bias and variance. An algorithm can't be more complex and less complex at the same time. For the graph, the perfect tradeoff will be like this.